Calculating Correct and Efficient Programs in Rocq

David Castro-Perez

David.CastroPerez@rhul.ac.uk

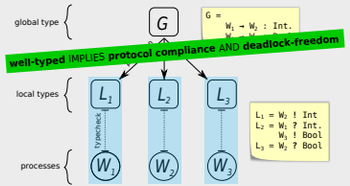


Based on joint work with Marco Paviotti and Michael Vollmer (University of Kent):
Program Optimisations via Hylomorphisms for Extraction of Executable Code, ITP 2025.

- Joint work with Marco Paviotti and Michael Vollmer (ITP 2025)
- Theme: fast *and* provably correct, without a new proof per program

My Research in One Slide

Protocols as types



well-typed ⇒ no deadlocks
POPL'19, OOPSLA'20,
ECOOP'23, POPL'26

Certified tools

$\Gamma \vdash M : T$

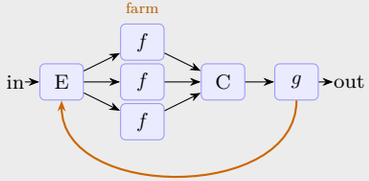
$h =_e f \circ \mathsf{fmap}_F\, h \circ g$

$(\nu a)\,(\overline{a}\langle 1\rangle \mid a(x).\,\overline{b}\langle x\rangle)$

Rocq / Agda / … ✓

mechanised metatheory,
certified languages
TACAS'20, PLDI'21, ITP'25

Parallelism



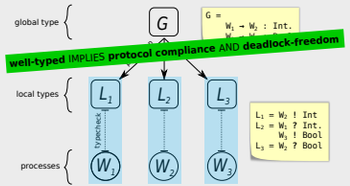
structured parallelism,
predicted speedups
ICFP'16, FGCS'18, CC'20

My Research in One Slide

- Point at left: protocols as types; well-typed means no deadlocks
- Middle: types, recursion laws, processes; all checked by a machine
- Right: task farms, pipelines, feedback; same results, predicted speedups
- Today: middle meets right

My Research in One Slide

Protocols as types



well-typed ⇒ no deadlocks
POPL'19, OOPSLA'20,
ECOOP'23, POPL'26

Certified tools

$$\Gamma \vdash M : T$$

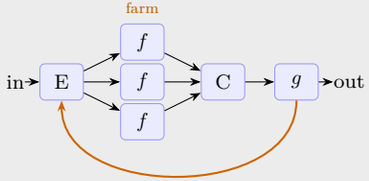
$$h =_e f \circ \mathsf{fmap}_F\, h \circ g$$

$$(\nu a)\,(\overline{a}\langle 1\rangle \mid a(x).\,\overline{b}\langle x\rangle)$$

Rocq / Agda / ... ✓

mechanised metatheory,
certified languages
TACAS'20, PLDI'21, ITP'25

Parallelism



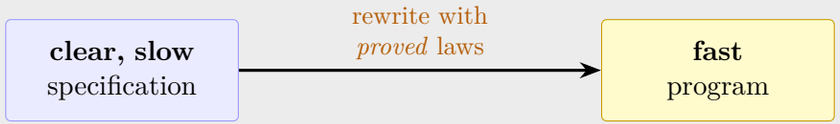
structured parallelism,
predicted speedups
ICFP'16, FGCS'18, CC'20

Today: certified tools + hylomorphisms → fast, correct code

My Research in One Slide

- Point at left: protocols as types; well-typed means no deadlocks
- Middle: types, recursion laws, processes; all checked by a machine
- Right: task farms, pipelines, feedback; same results, predicted speedups
- Today: middle meets right

In One Slide



The idea (1980s):

- Programs as algebra
- Optimise by rewriting
- Correct by construction

Our work:

- Every step machine-checked
- No extra axioms
- Runnable code out

In One Slide

- Point at the arrow: spec in, fast program out
- Each step is an equation, so the result equals the spec
- Old idea (Bird, Meertens); new: every step checked by Rocq, code you can run

The Problem

Fast *and* Correct

A Specification and a Program



A Specification and a Program

- Spec: sort, then double. Two passes, a throwaway list
- Program: doubles each pivot as it goes. One pass
- Equal? Ask the audience. Goal: derive it, not check it

A Specification and a Program




```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

A Specification and a Program

- Spec: sort, then double. Two passes, a throwaway list
- Program: doubles each pivot as it goes. One pass
- Equal? Ask the audience. Goal: derive it, not check it

A Specification and a Program




```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

Are they equal? Can we *derive* the second from the first?

A Specification and a Program

- Spec: sort, then double. Two passes, a throwaway list
- Program: doubles each pivot as it goes. One pass
- Equal? Ask the audience. Goal: derive it, not check it

Three Ways to Fast and Correct Code

	Proof effort	**Faster algorithm?**
Write, then **verify**	a new proof for every program	yes, if you write it

Three Ways to Fast and Correct Code

- Verify afterwards: works for anything, but a new proof each time
- Verified compilers (CompCert, CakeML): proved once, but keep your algorithm
- Calculation: laws proved once, reused; the program gets faster
- The catch: a library of laws general enough to apply

Three Ways to Fast and Correct Code

	Proof effort	**Faster algorithm?**
Write, then **verify**	a new proof for every program	yes, if you write it
Verified compiler	once, for the compiler	no: $O(n^3)$ stays $O(n^3)$

Three Ways to Fast and Correct Code

- Verify afterwards: works for anything, but a new proof each time
- Verified compilers (CompCert, CakeML): proved once, but keep your algorithm
- Calculation: laws proved once, reused; the program gets faster
- The catch: a library of laws general enough to apply

Three Ways to Fast and Correct Code

	Proof effort	**Faster algorithm?**
Write, then **verify**	a new proof for every program	yes, if you write it
Verified compiler	once, for the compiler	no: $O(n^3)$ stays $O(n^3)$
Calculate from a spec	once per law, then reused	yes, by rewriting

Three Ways to Fast and Correct Code

- Verify afterwards: works for anything, but a new proof each time
- Verified compilers (CompCert, CakeML): proved once, but keep your algorithm
- Calculation: laws proved once, reused; the program gets faster
- The catch: a library of laws general enough to apply

Three Ways to Fast and Correct Code

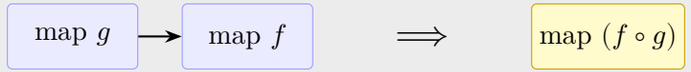
	Proof effort	**Faster algorithm?**
Write, then **verify**	a new proof for every program	yes, if you write it
Verified compiler	once, for the compiler	no: $O(n^3)$ stays $O(n^3)$
Calculate from a spec	once per law, then reused	yes, by rewriting

The catch: we need laws that *apply* to real programs. That is this talk.

Three Ways to Fast and Correct Code

- Verify afterwards: works for anything, but a new proof each time
- Verified compilers (CompCert, CakeML): proved once, but keep your algorithm
- Calculation: laws proved once, reused; the program gets faster
- The catch: a library of laws general enough to apply

You Already Trust Rewrite Rules

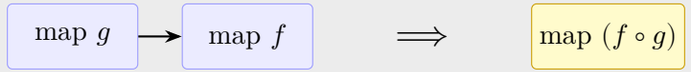


- ML compilers *fuse* tensor operations into one kernel.

You Already Trust Rewrite Rules

- Map fusion: two passes become one
- ML compilers fuse tensor ops (XLA, TVM)
- query optimisers
- compilers: inlining, loop fusion; GHC rewrite rules
- but: fixed rules, trusted; a wrong rule miscompiles silently

You Already Trust Rewrite Rules

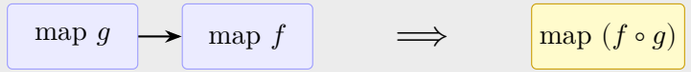


- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.

You Already Trust Rewrite Rules

- Map fusion: two passes become one
- ML compilers fuse tensor ops (XLA, TVM)
- query optimisers
- compilers: inlining, loop fusion; GHC rewrite rules
- but: fixed rules, trusted; a wrong rule miscompiles silently

You Already Trust Rewrite Rules

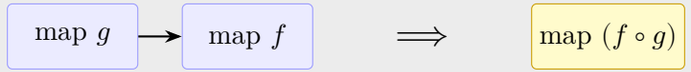


- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.
- Compilers inline, fuse loops, remove temporary data.

You Already Trust Rewrite Rules

- Map fusion: two passes become one
- ML compilers fuse tensor ops (XLA, TVM)
- query optimisers
- compilers: inlining, loop fusion; GHC rewrite rules
- but: fixed rules, trusted; a wrong rule miscompiles silently

You Already Trust Rewrite Rules



- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.
- Compilers inline, fuse loops, remove temporary data.
- Usually: a **fixed** set of rules, **trusted**, not proved.

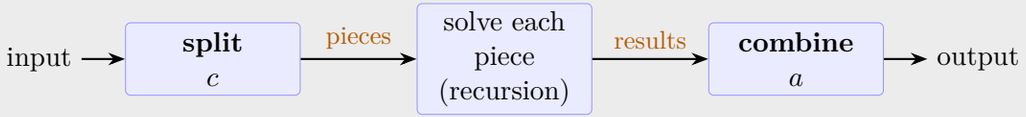
You Already Trust Rewrite Rules

- Map fusion: two passes become one
- ML compilers fuse tensor ops (XLA, TVM)
- query optimisers
- compilers: inlining, loop fusion; GHC rewrite rules
- but: fixed rules, trusted; a wrong rule miscompiles silently

The Laws

Divide and Conquer as Algebra

One Shape for Many Algorithms

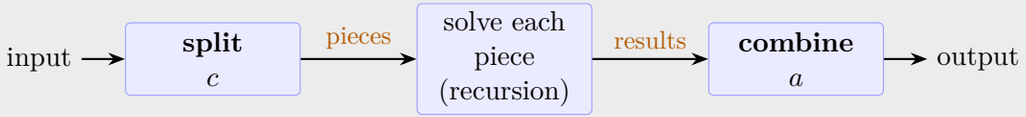


Algorithm	**split** c	**combine** a

One Shape for Many Algorithms

- Pipeline: split, recurse, combine
- quicksort: partition, concatenate
- mergesort: the work moves to combine
- power: split off the last bit
- recursion written once: the hylomorphism

One Shape for Many Algorithms

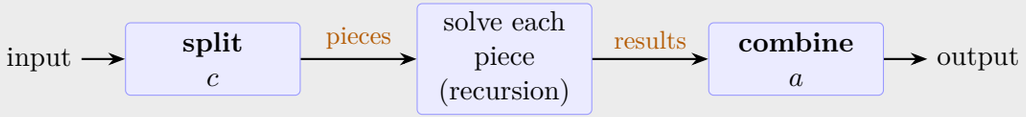


Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate

One Shape for Many Algorithms

- Pipeline: split, recurse, combine
- quicksort: partition, concatenate
- mergesort: the work moves to combine
- power: split off the last bit
- recursion written once: the hylomorphism

One Shape for Many Algorithms

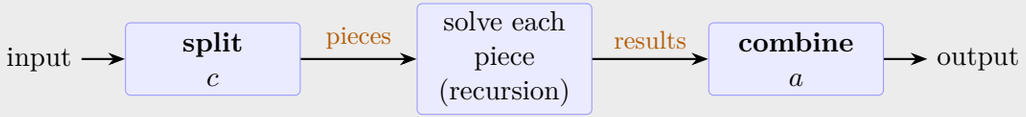


Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate
mergesort	cut in half	merge

One Shape for Many Algorithms

- Pipeline: split, recurse, combine
- quicksort: partition, concatenate
- mergesort: the work moves to combine
- power: split off the last bit
- recursion written once: the hylomorphism

One Shape for Many Algorithms

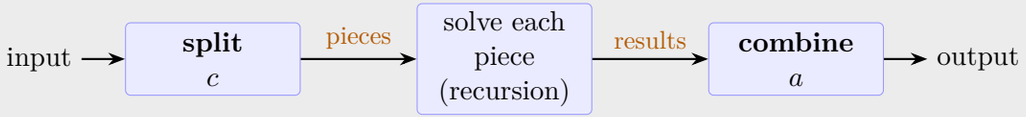


Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate
mergesort	cut in half	merge
x^n	$n \mapsto$ last bit, $n/2$	square, maybe multiply by x

One Shape for Many Algorithms

- Pipeline: split, recurse, combine
- quicksort: partition, concatenate
- mergesort: the work moves to combine
- power: split off the last bit
- recursion written once: the hylomorphism

One Shape for Many Algorithms



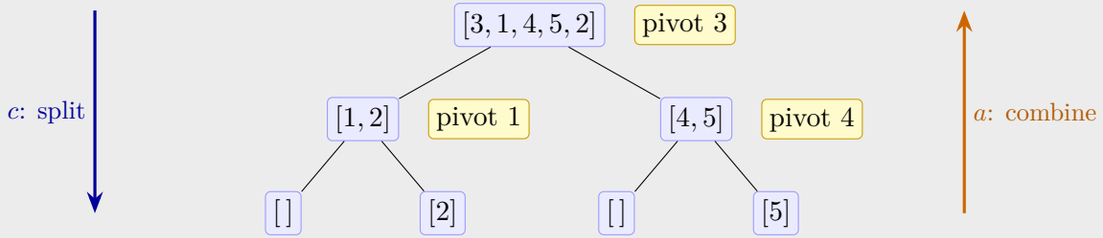
Algorithm	**split** c	**combine** a
quicksort	partition around a pivot	concatenate
mergesort	cut in half	merge
x^n	$n \mapsto$ last bit, $n/2$	square, maybe multiply by x

Hylomorphism $[\![a, c]\!]$: the recursion is *generic*; only a and c change.

One Shape for Many Algorithms

- Pipeline: split, recurse, combine
- quicksort: partition, concatenate
- mergesort: the work moves to combine
- power: split off the last bit
- recursion written once: the hylomorphism

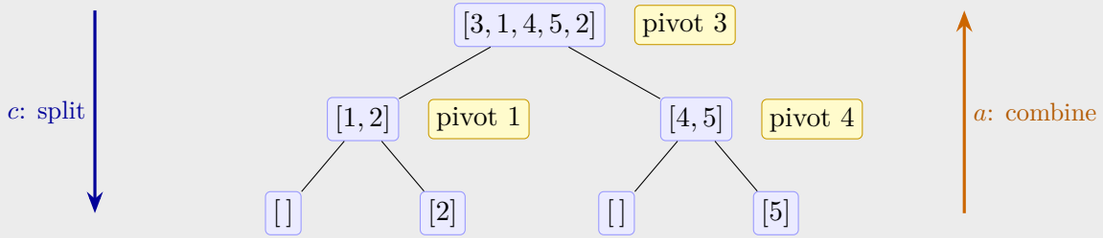
The Call Tree of Quicksort



The Call Tree of Quicksort

- Split goes down, combine comes up
- the tree is only the call stack, never in memory

The Call Tree of Quicksort

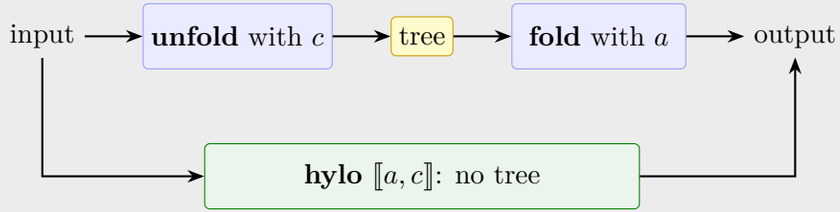


The tree is *virtual*: it is never built in memory.

The Call Tree of Quicksort

- Split goes down, combine comes up
- the tree is only the call stack, never in memory

Producers and Consumers

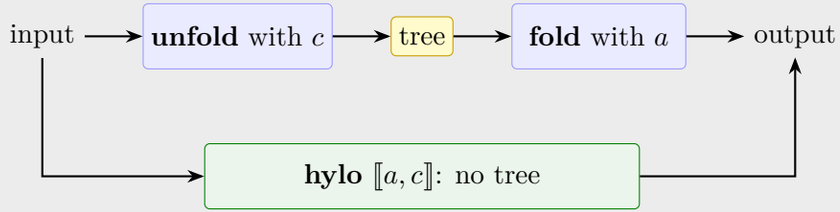


- Specification: *build*, then *consume*. Easy to read.

Producers and Consumers

- Top: build then consume; easy to read, wastes memory
- Bottom: hylo, no tree (deforestation, Wadler 1990)
- specs look like the top, programs like the bottom

Producers and Consumers



- Specification: *build*, then *consume*. Easy to read.
- Program: do both at once. Fast.

Producers and Consumers

- Top: build then consume; easy to read, wastes memory
- Bottom: hylo, no tree (deforestation, Wadler 1990)
- specs look like the top, programs like the bottom

Two Laws

Fusion $[\![a, c]\!] \rightarrow h = [\![b, c]\!]$

if h distributes over a

Deforestation $[\![\mathsf{build}, c]\!] \rightarrow [\![a, \mathsf{take_apart}]\!] = [\![a, c]\!]$

- Proved **once**, for every split, combine, data type.

Two Laws

- Fusion: push post-processing into combine, if it distributes
- Deforestation: build then take apart cancels out
- proved once, for every data type
- an optimisation is a chain of these

Two Laws

Fusion $[\![a, c]\!] \rightarrow h = [\![b, c]\!]$

if h distributes over a

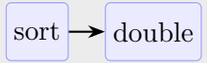
Deforestation $[\![\mathsf{build}, c]\!] \rightarrow [\![a, \mathsf{take_apart}]\!] = [\![a, c]\!]$

- Proved **once**, for every split, combine, data type.
- Each use is a *rewrite step*.

Two Laws

- Fusion: push post-processing into combine, if it distributes
- Deforestation: build then take apart cancels out
- proved once, for every data type
- an optimisation is a chain of these

Deriving the Example



Deriving the Example

- Quicksort = hylo with partition and concat
- doubling distributes over concat: fusion
- result: the hand-written program
- each step a theorem, applied in Rocq

Deriving the Example

$$\mathsf{sort} \to \mathsf{double}$$

$$= \quad [\![\mathsf{concat}, \mathsf{partition}]\!] \to \mathsf{double} \qquad \text{definition of quicksort}$$

Deriving the Example

- Quicksort = hylo with partition and concat
- doubling distributes over concat: fusion
- result: the hand-written program
- each step a theorem, applied in Rocq

Deriving the Example

$$\mathsf{sort} \rightarrow \mathsf{double}$$

$$= \quad [\![\mathsf{concat}, \mathsf{partition}]\!] \rightarrow \mathsf{double} \qquad \text{definition of quicksort}$$

$$= \quad [\![\mathsf{concat\ and\ double\ pivot}, \mathsf{partition}]\!] \qquad \text{fusion}$$

Deriving the Example

- Quicksort = hylo with partition and concat
- doubling distributes over concat: fusion
- result: the hand-written program
- each step a theorem, applied in Rocq

Deriving the Example

$$\text{sort} \to \text{double}$$

$$= \quad [\![\text{concat}, \text{partition}]\!] \to \text{double} \qquad \text{definition of quicksort}$$

$$= \quad [\![\text{concat and double pivot}, \text{partition}]\!] \qquad \text{fusion}$$

Each $=$ is a **theorem** applied by a proof assistant.

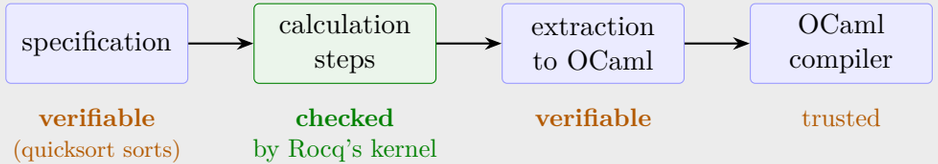
Deriving the Example

- Quicksort = hylo with partition and concat
- doubling distributes over concat: fusion
- result: the hand-written program
- each step a theorem, applied in Rocq

Machine-Checked

Hylomorphisms in Rocq

What You Have to Trust

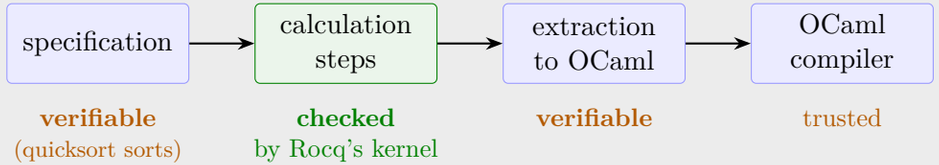


- Who picked the steps does not matter: a person, a script, a model.

What You Have to Trust

- Green box: checked by Rocq's small kernel
- Spec: verifiable; the library proves quicksort's output sorted and a permutation
- Extraction: verifiable; verified extraction to OCaml exists (MetaRocq, PLDI'24)
- Trusted: kernel, OCaml compiler
- steps can come from anyone: person, script, model
- wrong step = rejected

What You Have to Trust



- Who picked the steps does not matter: a person, a script, a model.
- A wrong step is **rejected**, not miscompiled.

What You Have to Trust

- Green box: checked by Rocq's small kernel
- Spec: verifiable; the library proves quicksort's output sorted and a permutation
- Extraction: verifiable; verified extraction to OCaml exists (MetaRocq, PLDI'24)
- Trusted: kernel, OCaml compiler
- steps can come from anyone: person, script, model
- wrong step = rejected

Three Obstacles

Termination
every function must provably stop

No extra axioms
comparing functions needs one

Clean code
generic proofs extract to unsafe casts

Three Obstacles

- Termination: Rocq only accepts recursion that provably stops
- Axioms: function equality needs one; we avoid it
- Clean code: textbook encoding extracts to unchecked casts (Obj.magic)
- Haskell has the laws, no termination

Three Obstacles

Termination
every function must provably stop

No extra axioms
comparing functions needs one

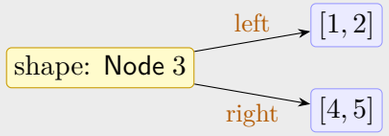
Clean code
generic proofs extract to unsafe casts

Haskell has the laws but not termination; Rocq has termination but rejects the textbook definitions.

Three Obstacles

- Termination: Rocq only accepts recursion that provably stops
- Axioms: function equality needs one; we avoid it
- Clean code: textbook encoding extracts to unchecked casts (Obj.magic)
- Haskell has the laws, no termination

Our Fix: Shapes and Positions



shape: Leaf no positions

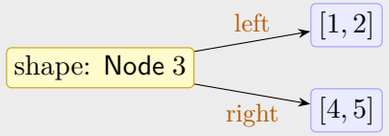
valid?	left	right
`Node h`	yes	yes
`Leaf`	no	no

- One type of positions + a **yes/no check**: plain OCaml types.

Our Fix: Shapes and Positions

- Data = shape + positions (containers)
- Point at the table: one position type, yes/no validity
- plain OCaml types
- proofs erased at extraction: no casts

Our Fix: Shapes and Positions



shape: Leaf no positions

valid?	left	right
`Node h`	yes	yes
`Leaf`	no	no

- One type of positions + a **yes/no check**: plain OCaml types.
- Proofs about the check are **erased** from the code.

Our Fix: Shapes and Positions

- Data = shape + positions (containers)
- Point at the table: one position type, yes/no validity
- plain OCaml types
- proofs erased at extraction: no casts

Termination: Recursion on a Proof



- Proved **once per split**, e.g. “pieces are shorter”.

Termination: Recursion on a Proof

- Proof tree has the same shape as the call tree; recurse on it
- one proof per split, e.g. pieces are shorter
- proof erased: code is just the recursion

Termination: Recursion on a Proof



- Proved **once per split**, e.g. “pieces are shorter”.
- The proof is scaffolding: **erased** at extraction.

Termination: Recursion on a Proof

- Proof tree has the same shape as the call tree; recurse on it
- one proof per split, e.g. pieces are shorter
- proof erased: code is just the recursion

No Extra Axioms

```
Print Assumptions mss_calc.
```
→ **Closed under the global context**

i.e. no axioms at all

- Equality = *same results on equal inputs* (setoids).

No Extra Axioms

- Rocq lists what a theorem depends on: here, nothing
- equality = same results on equal inputs
- laws usable as rewrite rules

No Extra Axioms

```
Print Assumptions mss_calc.
```
→ **Closed under the global context**

i.e. no axioms at all

- Equality = *same results on equal inputs* (setoids).
- Laws become *rewrite* steps.

No Extra Axioms

- Rocq lists what a theorem depends on: here, nothing
- equality = same results on equal inputs
- laws usable as rewrite rules

In Practice

From Calculation to Code

A Calculation in Rocq

```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e morph_of qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  rewrite <- (hylo_cps_hylo _ tsplit_k).
  finished.
Qed.
```

A Calculation in Rocq

- `Derive`: the program starts as an unknown
- the spec: quicksort, then double
- one line of laws; Rocq fills in the program

A Calculation in Rocq

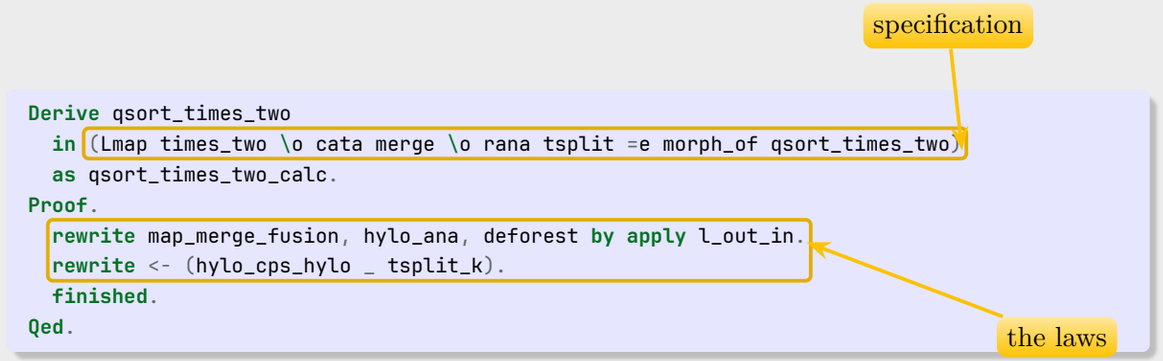
specification

```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e morph_of qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  rewrite <- (hylo_cps_hylo _ tsplit_k).
  finished.
Qed.
```

A Calculation in Rocq

- `Derive`: the program starts as an unknown
- the spec: quicksort, then double
- one line of laws; Rocq fills in the program

A Calculation in Rocq



A Calculation in Rocq

- `Derive`: the program starts as an unknown
- the spec: quicksort, then double
- one line of laws; Rocq fills in the program

Quicksort, Step by Step

```
(Lmap times_two \o cata merge) \o rana tsplit
```

Quicksort, Step by Step

- Each step: one law, the term Rocq shows after it
- map_merge_fusion: doubling moves inside merge
- hylo_ana, deforest: the tree between unfold and fold disappears (one step)
- hylo_cps_hylo: continuation-passing coalgebra, so the code has no closures

Quicksort, Step by Step

```
  (Lmap times_two \o cata merge) \o rana tsplit
=e  (* map_merge_fusion: double inside merge *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o rana tsplit
```

Quicksort, Step by Step

- Each step: one law, the term Rocq shows after it
- map_merge_fusion: doubling moves inside merge
- hylo_ana, deforest: the tree between unfold and fold disappears (one step)
- hylo_cps_hylo: continuation-passing coalgebra, so the code has no closures

Quicksort, Step by Step

```
  (Lmap times_two \o cata merge) \o rana tsplit
=e  (* map_merge_fusion: double inside merge *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o rana tsplit
=e  (* hylo_ana, deforest: unfold then fold, with no tree in between *)
  hylo (merge \o natural (nt_shape id times_two)) tsplit
```

Quicksort, Step by Step

- Each step: one law, the term Rocq shows after it
- map_merge_fusion: doubling moves inside merge
- hylo_ana, deforest: the tree between unfold and fold disappears (one step)
- hylo_cps_hylo: continuation-passing coalgebra, so the code has no closures

Quicksort, Step by Step

```
  (Lmap times_two \o cata merge) \o rana tsplit
=e  (* map_merge_fusion: double inside merge *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o rana tsplit
=e  (* hylo_ana, deforest: unfold then fold, with no tree in between *)
  hylo (merge \o natural (nt_shape id times_two)) tsplit
=e  (* hylo_cps_hylo: continuation-passing, for clean extraction *)
  hylo_cps (merge \o natural (nt_shape id times_two)) tsplit_k
```

Quicksort, Step by Step

- Each step: one law, the term Rocq shows after it
- map_merge_fusion: doubling moves inside merge
- hylo_ana, deforest: the tree between unfold and fold disappears (one step)
- hylo_cps_hylo: continuation-passing coalgebra, so the code has no closures

Quicksort: What Comes Out

```
let rec qsort_times_two = function
| [] -> []
| i :: l ->
  let (l0, l1) = partition (fun x0 -> Uint63.lt x0 i) l in
  app (qsort_times_two l0)
    ((Uint63.mul (Uint63.of_int (2)) i) :: (qsort_times_two l1))
```

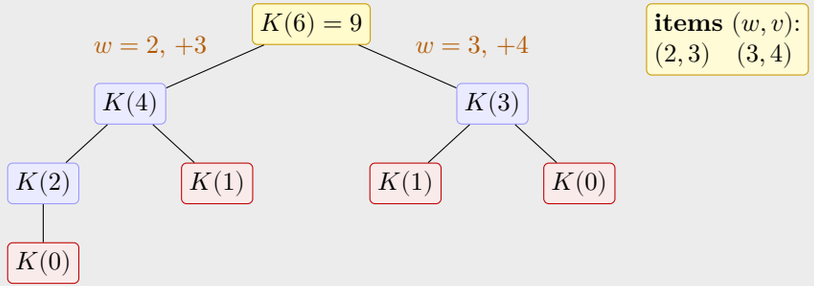
The last line, extracted: **one pass**, doubling each pivot as it goes. The hand-written program from the start, now with a proof.

Quicksort: What Comes Out

- Real extraction output of the last line
- Point at the pivot: `mul 2 i`, done in place
- Same as the hand-written version; now proved equal to sort-then-double

Dynamic Programming: Knapsack

$$K(0) = 0 \qquad K(c) = \max\left(\{0\} \cup \{\, K(c - w_i) + v_i \mid 0 < w_i \le c \,\}\right)$$

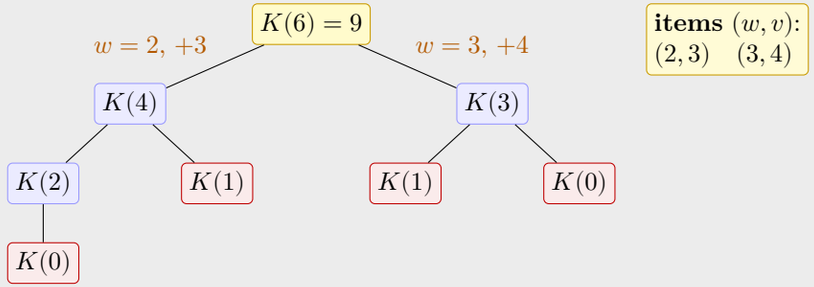


Dynamic Programming: Knapsack

- Classic DP recurrence; items (2, 3) and (3, 4), capacity 6
- Each branch takes one item: w off the capacity, v added
- Point at red nodes: the same capacities computed again and again
- naive recursion is a hylo, but exponential

Dynamic Programming: Knapsack

$$K(0) = 0 \qquad K(c) = \max\left(\{0\} \cup \{\, K(c - w_i) + v_i \mid 0 < w_i \le c \,\}\right)$$

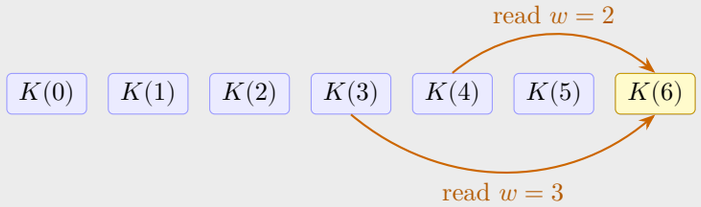


A hylomorphism, but it **recomputes**: exponential.

Dynamic Programming: Knapsack

- Classic DP recurrence; items (2, 3) and (3, 4), capacity 6
- Each branch takes one item: w off the capacity, v added
- Point at red nodes: the same capacities computed again and again
- naive recursion is a hylo, but exponential

Remember, Don't Recompute



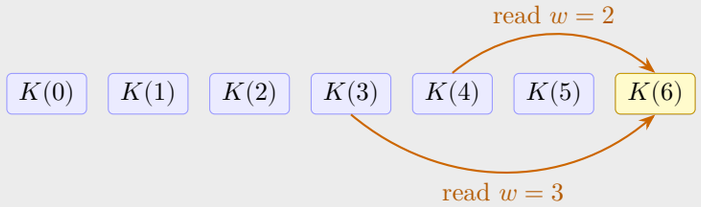
- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

Remember, Don't Recompute

- Keep a table of results for every capacity
- Point at arrows: K(6) reads K(4) and K(3) from the table
- dynamorphism (Hinze, Wu, Gibbons)
- law (`reenumerate`): change what we recurse on, items to capacities
- exponential to one entry per capacity

Remember, Don't Recompute



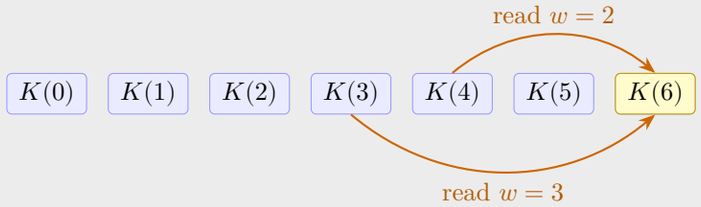
- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.
- A law: recursion over *items* = recursion over *capacities* with a table, if each step *looks up* its subproblems.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

Remember, Don't Recompute

- Keep a table of results for every capacity
- Point at arrows: K(6) reads K(4) and K(3) from the table
- dynamorphism (Hinze, Wu, Gibbons)
- law (`reenumerate`): change what we recurse on, items to capacities
- exponential to one entry per capacity

Remember, Don't Recompute



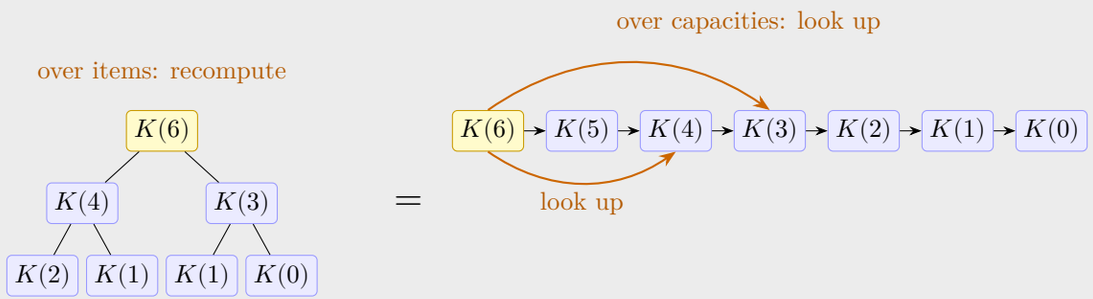
- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.
- A law: recursion over *items* = recursion over *capacities* with a table, if each step *looks up* its subproblems.
- Exponential $\Longrightarrow$ one entry per capacity.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

Remember, Don't Recompute

- Keep a table of results for every capacity
- Point at arrows: K(6) reads K(4) and K(3) from the table
- dynamorphism (Hinze, Wu, Gibbons)
- law (`reenumerate`): change what we recurse on, items to capacities
- exponential to one entry per capacity

From Recomputation to a Table, by Rewriting



From Recomputation to a Table, by Rewriting

- Left: naive recursion over items; repeats
- Right: count down capacities; each entry looks up earlier ones
- `reenumerate`: switch the recursion; array tables, O(1) lookups
- `dyna_loop`: the table is built by a loop
- Side goals: the loop's step, its first entry, the lookup algebra
- Exponential to linear in the capacity; no index arithmetic by hand

From Recomputation to a Table, by Rewriting

```
Derive knapsack_arr in (knapsack_naive wvs =e morph_of knapsack_arr)
  as knapsack_arr_calc.
Proof.
  unfold knapsack_naive.
  (* Memoise: the subproblems are read from a table *)
  erewrite reenumerate.
  (* The table of a count-down is built by a loop *)
  erewrite dyna_loop.
  (* The loop's step, its first entry, and the algebra on the table *)
  2: exact knap_step_calc.
  2: exact knap_base.
  2: exact knap_look.
  finished.
Qed.
```

From Recomputation to a Table, by Rewriting

- Left: naive recursion over items; repeats
- Right: count down capacities; each entry looks up earlier ones
- `reenumerate`: switch the recursion; array tables, O(1) lookups
- `dyna_loop`: the table is built by a loop
- Side goals: the loop's step, its first entry, the lookup algebra
- Exponential to linear in the capacity; no index arithmetic by hand

From Recomputation to a Table, by Rewriting



From Recomputation to a Table, by Rewriting

- Left: naive recursion over items; repeats
- Right: count down capacities; each entry looks up earlier ones
- `reenumerate`: switch the recursion; array tables, O(1) lookups
- `dyna_loop`: the table is built by a loop
- Side goals: the loop's step, its first entry, the lookup algebra
- Exponential to linear in the capacity; no index arithmetic by hand

Knapsack, Step by Step

```
knapsack_naive wvs
```

Knapsack, Step by Step

- The goal after each rewrite, as Rocq prints it
- reenumerate: from items to capacities, lookups jump into the array, O(1)
- dyna_loop: the table is built by a loop counting up
- Last line: a loop over the capacities, one step each

Knapsack, Step by Step

```
  knapsack_naive wvs
=e  (* unfold: the exponential specification *)
  hylo naive_alg (naive_c wvs)
```

Knapsack, Step by Step

- The goal after each rewrite, as Rocq prints it
- reenumerate: from items to capacities, lookups jump into the array, O(1)
- dyna_loop: the table is built by a loop counting up
- Last line: a loop over the capacities, one step each

Knapsack, Step by Step

```
  knapsack_naive wvs
=e  (* unfold: the exponential specification *)
  hylo naive_alg (naive_c wvs)
=e  (* reenumerate: recurse on capacities, look results up in O(1) *)
  dyna (knap_alg wvs \o look_with (w:=fast_drop) fast_drop_walk) out_int_c
```

Knapsack, Step by Step

- The goal after each rewrite, as Rocq prints it
- reenumerate: from items to capacities, lookups jump into the array, O(1)
- dyna_loop: the table is built by a loop counting up
- Last line: a loop over the capacities, one step each

Knapsack, Step by Step

```
  knapsack_naive wvs
=e  (* unfold: the exponential specification *)
  hylo naive_alg (naive_c wvs)
=e  (* reenumerate: recurse on capacities, look results up in O(1) *)
  dyna (knap_alg wvs \o look_with (w:=fast_drop) fast_drop_walk) out_int_c
=e  (* dyna_loop: the table of a count-down is built by a loop *)
  (mhead \o l_out) \o
    (morphism fun n : int =>
       loop n (fun _ : int => memo_step (knap_step wvs))
         (0%uint63, memo_base 0%uint63))
```

Knapsack, Step by Step

- The goal after each rewrite, as Rocq prints it
- reenumerate: from items to capacities, lookups jump into the array, O(1)
- dyna_loop: the table is built by a loop counting up
- Last line: a loop over the capacities, one step each

Knapsack: What Comes Out

```
let knap_step items t =                    (* best over the items *)
  max 0 (max_list (List.map (fun (w, v) ->
    match fast_drop w t with               (* jump into the array: O(1) *)
    | Some a -> head a + v
    | None -> min_int) items))

let knapsack items c =                     (* a loop over the capacities *)
  let rec loop i t =
    if i < c then loop (i + 1) (push (knap_step items t) t) else t
  in head (loop 0 (single 0))
```

(simplified)

A **loop** over the capacities; each step looks up $K(c-w)$ in $O(1)$. Linear in the capacity.

Knapsack: What Comes Out

- Simplified from the extracted code; the real one is in the library
- `knapsack`: a loop counting capacities up; `push` adds one entry
- `knap_step`: best over the items; `fast_drop` jumps into the array
- Exponential spec to a linear loop, every step proved

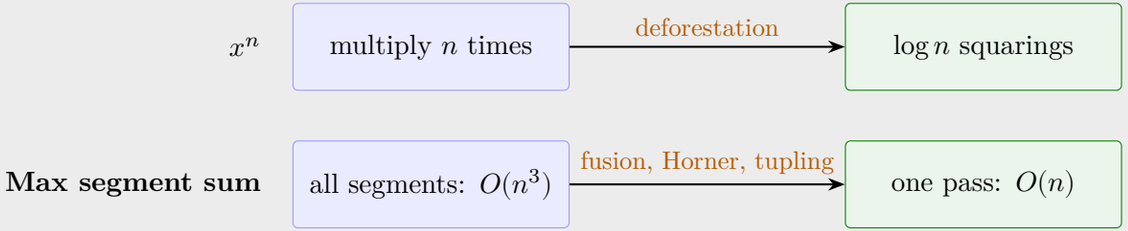
More Calculations



More Calculations

- Fast power: deforest the list of binary digits
- max segment sum: Bird's classic, cubic to linear

More Calculations



More Calculations

- Fast power: deforest the list of binary digits
- max segment sum: Bird's classic, cubic to linear

Maximum Segment Sum: Spec and Result

$$\mathsf{mss} = \mathsf{maximum} \circ \mathsf{map}\,\mathsf{sum} \circ \mathsf{segments}$$

3	−5	2	4	−1

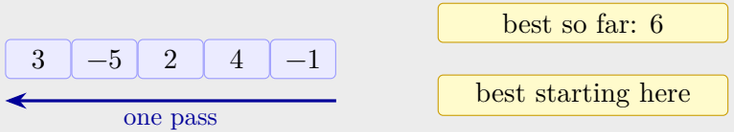
best segment: $2 + 4 = 6$

Maximum Segment Sum: Spec and Result

- Spec is the definition: all segments, sum, max
- Calculated: one pass, two accumulators; not obviously correct, but proved
- Bird's classic, machine-checked; steps are library laws

Maximum Segment Sum: Spec and Result

$$\mathsf{mss} = \mathsf{maximum} \circ \mathsf{map}\,\mathsf{sum} \circ \mathsf{segments}$$



Calculated: cubic $\Longrightarrow$ **one pass**, two numbers.

Maximum Segment Sum: Spec and Result

- Spec is the definition: all segments, sum, max
- Calculated: one pass, two accumulators; not obviously correct, but proved
- Bird's classic, machine-checked; steps are library laws

Summary So Far

- A **library**: hylomorphisms and their laws in Rocq.
- Optimisations are **proofs by rewriting**.
- Termination proved **once per split**.
- **No extra axioms**; OCaml **without unsafe casts**.

D. Castro-Perez, M. Paviotti, M. Vollmer. **Program Optimisations via Hylomorphisms for Extraction of Executable Code**. ITP 2025. Library: `https://github.com/dcastrop/rocq-hylomorphisms`

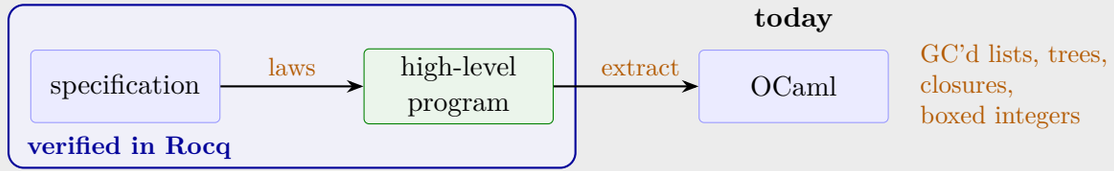
Summary So Far

- The library today: ITP 2025 plus later work
- Next: work in progress

Work in Progress

Down to Low-Level Code

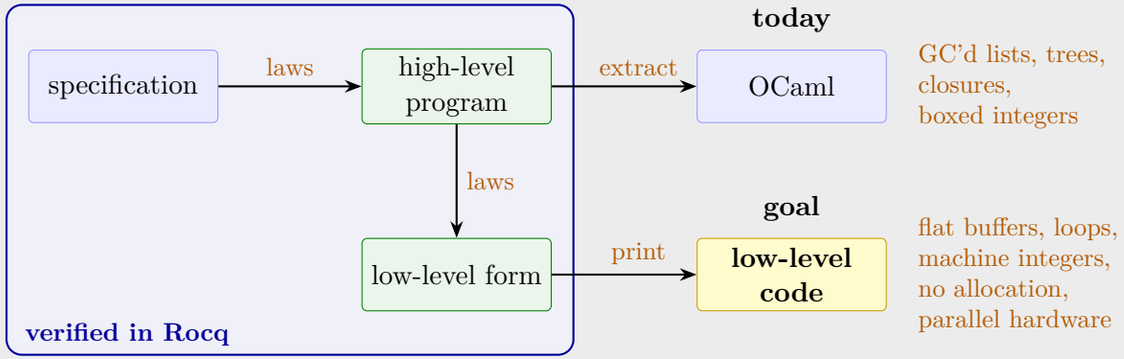
Two Ways Out of Rocq



Two Ways Out of Rocq

- Box = verified in Rocq
- Today: exit through OCaml (GC, closures, boxed integers)
- Goal: more laws to a low-level form, printed directly
- layout and scheduling become calculation steps, not a compiler pass

Two Ways Out of Rocq



Two Ways Out of Rocq

- Box = verified in Rocq
- Today: exit through OCaml (GC, closures, boxed integers)
- Goal: more laws to a low-level form, printed directly
- layout and scheduling become calculation steps, not a compiler pass

What We Are Working On

Low-level, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

- You choose *laws*, not pointers and indices.

What We Are Working On

- Read the goal box
- you choose laws, not indices
- every step proved
- cost model picks among equal programs

What We Are Working On

Low-level, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

- You choose *laws*, not pointers and indices.
- Every step proved; the output is machine-level code.

What We Are Working On

- Read the goal box
- you choose laws, not indices
- every step proved
- cost model picks among equal programs

What We Are Working On

Low-level, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

- You choose *laws*, not pointers and indices.
- Every step proved; the output is machine-level code.
- A cost model says *which* equal program to pick.

What We Are Working On

- Read the goal box
- you choose laws, not indices
- every step proved
- cost model picks among equal programs

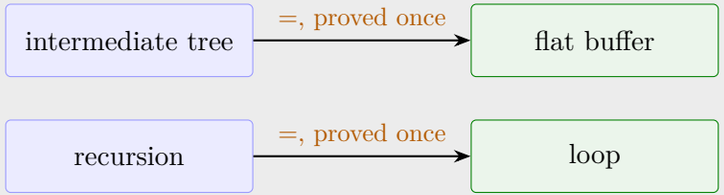
Ongoing: Laws That Change Representation



Ongoing: Laws That Change Representation

- Today's laws only know trees
- recursion to loops
- independent pieces run in parallel
- all proved once, used as rewrites

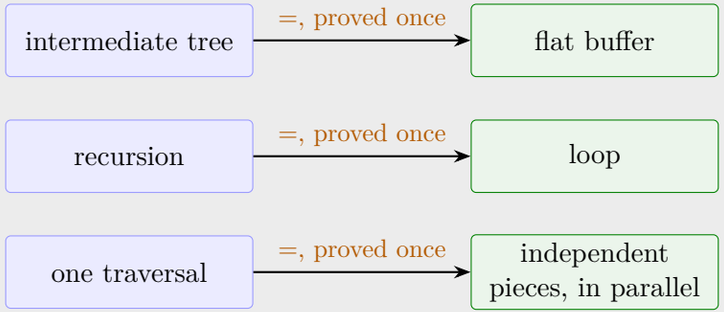
Ongoing: Laws That Change Representation



Ongoing: Laws That Change Representation

- Today's laws only know trees
- recursion to loops
- independent pieces run in parallel
- all proved once, used as rewrites

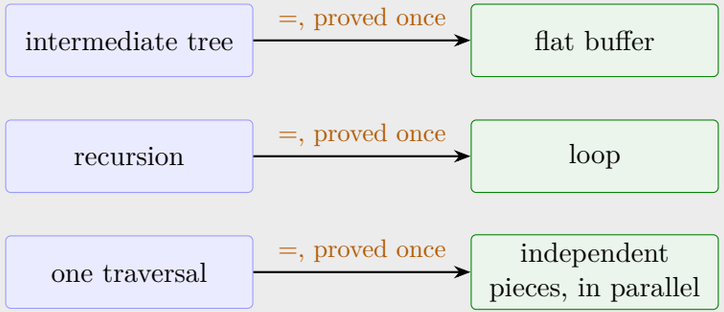
Ongoing: Laws That Change Representation



Ongoing: Laws That Change Representation

- Today's laws only know trees
- recursion to loops
- independent pieces run in parallel
- all proved once, used as rewrites

Ongoing: Laws That Change Representation

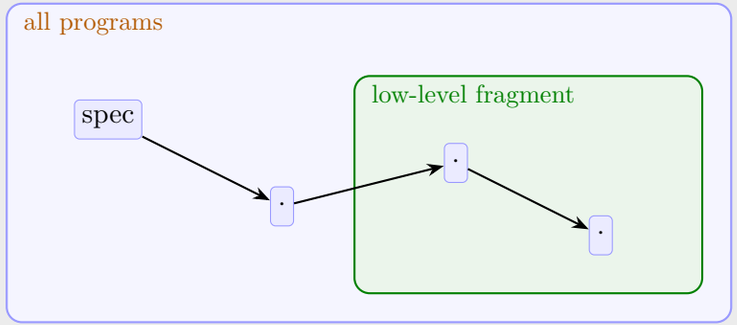


Today's laws change the *algorithm*; these change *where data lives* and *how work is scheduled*.

Ongoing: Laws That Change Representation

- Today's laws only know trees
- recursion to loops
- independent pieces run in parallel
- all proved once, used as rewrites

Ongoing: A Target the Laws Never Leave

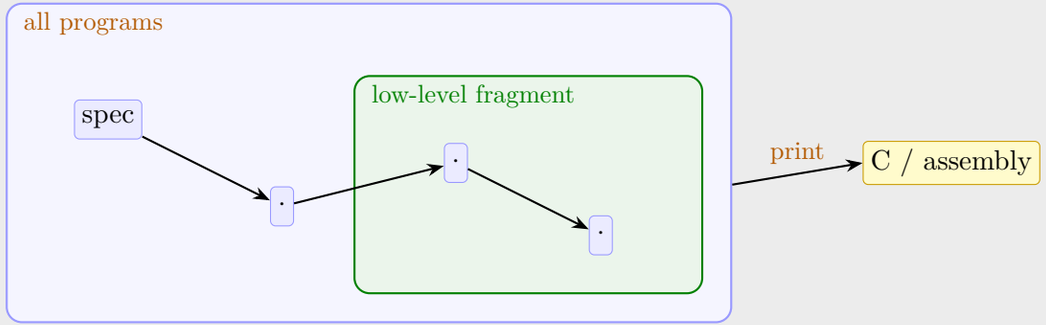


- Every law keeps programs **inside** the fragment.

Ongoing: A Target the Laws Never Leave

- Low-level fragment: no closures, explicit memory, machine integers
- Laws keep programs inside it
- printing is almost syntactic: small trusted base
- overflow and bounds as side conditions

Ongoing: A Target the Laws Never Leave

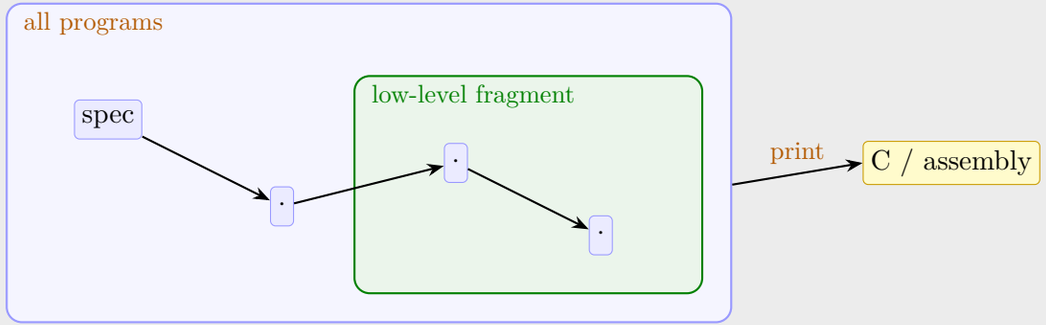


- Every law keeps programs **inside** the fragment.
- Only a **small printer** is trusted.

Ongoing: A Target the Laws Never Leave

- Low-level fragment: no closures, explicit memory, machine integers
- Laws keep programs inside it
- printing is almost syntactic: small trusted base
- overflow and bounds as side conditions

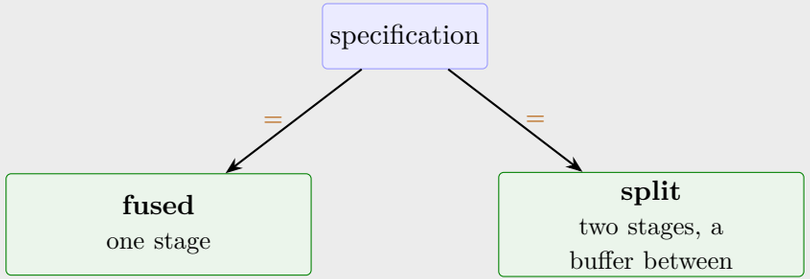
Ongoing: A Target the Laws Never Leave



- Every law keeps programs **inside** the fragment.
- Only a **small printer** is trusted.
- Overflow and bounds: **side conditions** of the laws.

Ongoing: A Target the Laws Never Leave

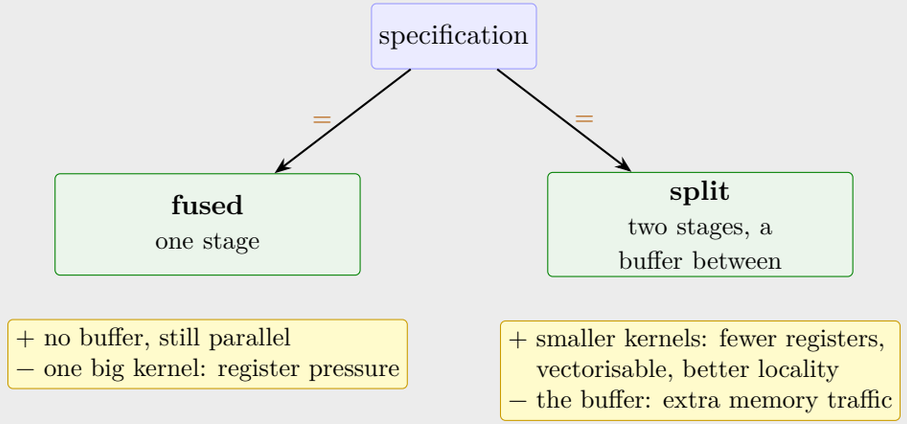
- Low-level fragment: no closures, explicit memory, machine integers
- Laws keep programs inside it
- printing is almost syntactic: small trusted base
- overflow and bounds as side conditions



Ongoing: A Cost Model

- Two equal programs from one spec
- fused: no buffer, still parallel; but one big kernel, register pressure
- split: smaller kernels (registers, occupancy, vectorisation, locality); but buffer traffic
- which wins depends on the machine and the input size
- laws say equal, cost model says better

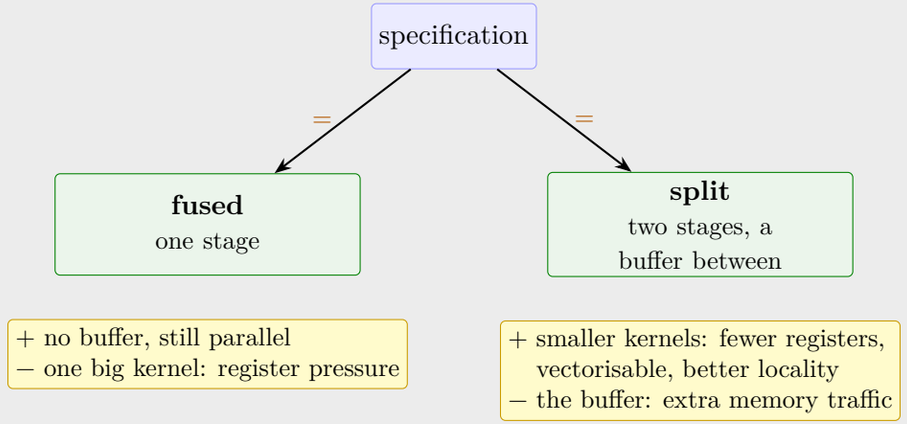
Ongoing: A Cost Model



Ongoing: A Cost Model

- Two equal programs from one spec
- fused: no buffer, still parallel; but one big kernel, register pressure
- split: smaller kernels (registers, occupancy, vectorisation, locality); but buffer traffic
- which wins depends on the machine and the input size
- laws say equal, cost model says better

Ongoing: A Cost Model

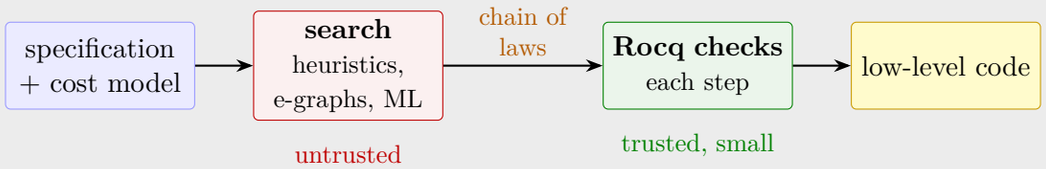


The laws say what is *equal*; a cost model says what is *better*.

Ongoing: A Cost Model

- Two equal programs from one spec
- fused: no buffer, still parallel; but one big kernel, register pressure
- split: smaller kernels (registers, occupancy, vectorisation, locality); but buffer traffic
- which wins depends on the machine and the input size
- laws say equal, cost model says better

Untrusted Search, Trusted Checking

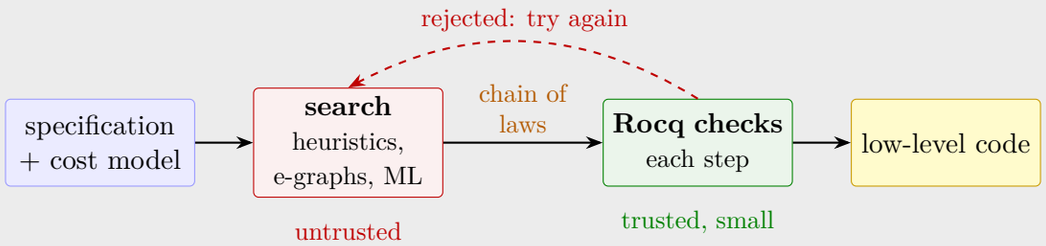


- The derivation is a **certificate**.

Untrusted Search, Trusted Checking

- Search: heuristics, e-graphs, or an ML model
- Rocq checks the chain of laws
- wrong proposals rejected; output correct regardless
- ML: like a model proposing proofs; security: small trusted base

Untrusted Search, Trusted Checking



- The derivation is a **certificate**.
- Search can be as clever, or as wrong, as it likes.

Untrusted Search, Trusted Checking

- Search: heuristics, e-graphs, or an ML model
- Rocq checks the chain of laws
- wrong proposals rejected; output correct regardless
- ML: like a model proposing proofs; security: small trusted base

Where We Are



exactly covered, no overlaps

In Rocq, axiom-free:

- Splitting index ranges k ways
- Machine integers, no overflow (proved)
- Buffers, compared by their reads
- Tree reductions over ranges
- Writes to a buffer, driven by any split

Not yet:

- Laws that add or remove a buffer
- Each index written exactly once
- The low-level target and printer
- The cost model and search

Where We Are

- Early stage; be honest
- Done: exact-cover splits, machine integers, buffers, tree reductions, writes
- Not yet: buffer laws, exactly-once, target, cost model

Summary

- Program optimisation as **algebra**: rewrite with laws.
- Divide and conquer as **hylomorphisms**: laws proved once.
- In Rocq: **machine-checked**, no extra axioms, clean OCaml.
- Ongoing: **low-level, correct-by-construction code** from high-level steps, guided by a **cost model**.

Summary

- Recap the four bullets

Thank you!

Questions?

`https://github.com/dcastrop/rocq-hylomorphisms`

Based on joint work with Marco Paviotti and Michael Vollmer.

- Thank Marco and Michael; questions